Python Engineering

Class 05 - Lists & Tuples

Referencias, mutabilidad y por que trajectory cambiaba solo

Nota de apoyo para **week_02_data_structures -> class_05_lists_tuples**. Este documento explica el comportamiento observado al guardar listas dentro de una tupla en `trajectory`.

1. El comportamiento observado

El problema aparecia al crear tres listas para almacenar la historia de altura, velocidad y tiempo, y despues guardar esas listas dentro de trajectory.

```
h = []
v = []
t = []

h.append(h0)
v.append(v0)
t.append(t0)

trajectory.append((h, v, t))
```

> Ese ultimo append no guarda los escalares iniciales. Guarda una tupla que contiene **referencias a las tres listas**.

2. Referencia, no fotografia

```
h ----------------------> [0]
                           ^
trajectory[0][0] ----------|

v ----------------------> [50]
trajectory[0][1] ----------|

t ----------------------> [0]
trajectory[0][2] ----------|
```

Cuando despues ejecutas `h.append(h_new)`, no se crea una lista nueva. Se modifica el mismo objeto al que ya apunta trajectory.

3. Por eso el primer elemento crecia

```
h.append(5.0)
v.append(49.019)
t.append(0.1)
```

Las listas pasan a contener varios valores, y como trajectory conserva referencias a ellas, al imprimir trajectory se ve algo como:

```
[
  ([0, 5.0, 9.90, ...],
   [50, 49.019, 48.038, ...],
   [0, 0.1, 0.2, ...]),

  (5.0, 49.019, 0.1),
  (9.90, 48.038, 0.2),
  ...
]
```

4. Pero la tupla no era inmutable?

Si. La tupla es inmutable en cuanto a su propia estructura: no puedes reemplazar libremente sus elementos. Pero una tupla puede contener objetos mutables, como listas, y esos objetos si pueden modificarse.

```
my_list = [1]
my_tuple = (my_list,)

my_list.append(2)

print(my_tuple)
# ([1, 2],)
```

Idea clave: tupla inmutable no significa contenido profundamente inmutable.

5. El nombre tecnico: aliasing

Cuando dos nombres o estructuras diferentes apuntan al mismo objeto mutable, se produce **aliasing**. Cambiar el objeto desde una referencia hace que el cambio sea visible desde todas las demas referencias.

6. Lo que querias guardar realmente

Cada elemento de trajectory debe representar un **estado escalar completo** de la simulacion, no tres historiales completos.

```
trajectory.append((t0, h0, v0))
```

y dentro del loop:

```
trajectory.append((t_new, h_new, v_new))
```

Entonces trajectory queda conceptualmente como una lista de estados:

```
trajectory = [
    (0.0, 0.0, 50.0),
    (0.1, 5.0, 49.019),
    (0.2, 9.9019, 48.038),
    ...
]
```

7. Dos representaciones validas de la misma trayectoria

Tres listas por variable	Lista de tuplas por estado
times = [...] heights = [...] velocities = [...]	trajectory = [(t0,h0,v0), (t1,h1,v1), ...]
Muy comoda para graficas y operar por variable.	Muy comoda para trabajar con estados completos por epoca.

8. Y si quisieras guardar una copia de las listas?

Si realmente quisieras congelar el contenido de las listas en un instante determinado, podrias crear copias:

```
trajectory.append((h.copy(), v.copy(), t.copy()))
```

Pero esa no es la estructura deseada en EX13, porque cada elemento deberia representar un estado `(t, h, v)`, no un historial entero.

9. Version recomendada para EX13

```
h = []
v = []
t = []
trajectory = []

h.append(h0)
v.append(v0)
t.append(t0)
trajectory.append((t0, h0, v0))

while h[i] >= 0:
```

```
    h_new, v_new, t_new = euler_step(h[i], v[i], t[i], g, dt)

    h.append(h_new)
    v.append(v_new)
    t.append(t_new)
    trajectory.append((t_new, h_new, v_new))

    i += 1
```

10. Learning Tracker

Concepto	Estado
Lista como objeto mutable	Aprendido
Tupla como estructura inmutable	Aprendido
Objeto mutable dentro de una tupla	Aprendido
Referencia vs copia	Aprendido
Aliasing	Introducido
Lista de variables vs lista de estados	Aprendido
copy()	Introducido

Frase para recordar: Python no guarda una fotografia de una lista cuando la insertas en otra estructura; normalmente guarda una referencia al mismo objeto. Si ese objeto mutable cambia, el cambio se ve desde todas sus referencias.